

Docker Networking

Bridge · Host · Overlay · DNS · Port Mapping

■ Analogy

- Docker networks are like office buildings. Bridge network = offices in the same building (talk via hallways). Host network = working from home with your own door to the street. Overlay network = two buildings connected by a secure tunnel. Each container is an office — Docker's DNS is the building directory.

Key Terms

Bridge network	Default network. Containers get private IPs; communicate by container name within same bridge
Host network	Container shares host's network stack — no isolation. Best for performance-critical tools
Overlay network	Spans multiple Docker hosts (Swarm). Containers on different machines communicate transparently
Macvlan	Assigns a MAC address to container — appears as physical device on network
None network	Completely isolated — no network interface except loopback
Port mapping (-p)	-p hostPort:containerPort — forwards traffic from host into container
Embedded DNS	Docker's built-in DNS resolves container names → IPs within the same user-defined bridge
docker0	Default bridge interface created by Docker on the host
CNI	Container Network Interface — plugin spec. Docker uses its own; Kubernetes uses CNI plugins
Network alias	--network-alias — assign a DNS alias to a container on a network

Network Types Comparison

Type	Isolation	DNS by Name	Use Case
bridge (default)	Container-level	No (use user-defined)	Single host, quick testing
user-defined bridge	Container-level	Yes	Multi-container apps on one host
host	None	N/A	Performance, monitoring agents
overlay	Container-level	Yes	Docker Swarm, multi-host

macvlan	None	No	Legacy apps needing MAC address
none	Full isolation	No	Security-sensitive workloads

Networking Commands

```
# Create a user-defined bridge (enables DNS by name)
docker network create mynet

# Run containers on the same network
docker run -d --name db --network mynet postgres:15
docker run -d --name app --network mynet -e DB_HOST=db myapp

# app can reach db at hostname 'db' — Docker DNS resolves it

# Expose port 8080 on host → 80 in container
docker run -d -p 8080:80 --network mynet nginx

# Inspect network (see IPs, containers)
docker network inspect mynet

# Connect existing container to another network
docker network connect mynet existing-container

# Remove unused networks
docker network prune
```

■ ■ Default Bridge vs User-Defined Bridge

- Default bridge (docker0): containers can only communicate by IP — no DNS resolution by name
- User-defined bridge: containers resolve each other by container name via embedded DNS
- Always create a user-defined bridge for multi-container apps — don't rely on `--link` (deprecated)

■ Real-World: Airbnb

- Airbnb isolates each service in its own Docker network, allowing db, cache, and app containers to communicate by name while keeping cross-service traffic segregated. This mirrors their microservices security model — services cannot accidentally reach each other's databases.